Laboratório de Concorrência · Aula 10

Matheus De Oliveira Mascarello · RA 241207402 · Turma A
Gerado em 09/10/2026, 21:11:53 · tempo registrado no cronômetro: 0:00

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

A diferença é que, em cooperação participantes trabalham juntos para alcançar um objetivo comum, ajudando uns aos outros e coordenando suas ações. Já em competição os participantes coordenam suas ações porém estão competindo entre si para alcançar um objetivo individual.
Exemplos:
Cooperação -> Carregar um sofá em grupo.
Competição -> Corrida de atletismo.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

Esperamos que imprima 2.000.0000

1ª execução (n = 1.000.000)

1032817

2ª execução (n = 1.000.000)

1340254

3ª execução (n = 1.000.000)

1329380

Saída do programa

```
esperado: 2000000
obtido:   1329380
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não, os valores não foram iguais e também não deram o resultado esperado. Por conta de que há algum erro acontecendo não está totalmente organizado fazendo com que as threads contem ao mesmo tempo, e as vezes contem em tempos diferentes variando o resultado.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

a thread lê, soma e grava após. porém quando temos duas threads fazendo isso simultaneamente no mesmo contador, pode ocorrer de fazerem ao mesmo tempo ou seja: a thread "A" lê 1, soma 1 e grava, e a thread "B" fez isso ao mesmo tempo então ela também leu 1, somou 1, e gravou 1. dando o mesmo resultado que a Thread "A", não cooperaram entre si.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com n = 1000 alcançou o resultado esperado. Quer dizer que ele consegue até alcançar o numero se for um numero baixo de contagens, porém em um numero alto ele irá falhar.
Em um sistema real onde milhões de pessoas acessam, ele não conseguirá desempenhar sua função, ocasionando erros.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

ABABAB → 6

ABBABA → 4

BBBAAA → 7

AABBBA → 4

Saída do programa

```
print("Ordens possíveis:", sum(len(v) for v in resultados.values()))
for valor in sorted(resultados):
    ordens = resultados[valor]
    print(f"TOTAL = {valor}: {len(ordens):2d} ordens, por exemplo {ordens[0]}")
139 ms
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4:  9 ordens, por exemplo AABBBA
TOTAL = 6:  9 ordens, por exemplo AABABB
TOTAL = 7:  1 ordens, por exemplo BBBAAA
TOTAL = 8:  1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Ordens possíveis: 20
TOTAL = 8: 1 ordens, por exemplo AAABBB

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

por exemplo AABBBA,
O A já inicia a leitura, e faz a soma porém ainda não grava o resultado. O B interrompe o A e faz os calculos e grava o resultado, Logo após o A grava o resultado que tinha calculado, substituindo assim o resultado de B.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acredito que a Operação atômica seja mais rápida. Acredito que haja possibilidade do Monitor falhar.

Saída do programa

```
esperado:          2000000
sem sincronizacao: 1025861 em 2 ms
```

```
synchronized:      2000000 em 79 ms
AtomicInteger:     2000000 em 27 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

synchronized: 2000000 em 79 ms
AtomicInteger: 2000000 em 27 ms.
Pois há regras a serem seguidas, passo a passo. tornando mais lenta a execução dos que possuem algum tipo de regra de sincronização.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não. Um AtomicInteger em cada saldo, sozinho, não garante que uma transferência bancária seja segura. Isso acontece porque uma transferência envolve duas operações relacionadas: Retirar dinheiro da conta de origem, Depositar dinheiro na conta de destino.
não garante que as duas operações da transferência aconteçam como uma única operação indivisível.
o AtomicInteger protege cada saldo individualmente, mas uma transferência exige proteger a operação completa, envolvendo as duas contas, para manter a consistência dos valores.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Acredito que não, pois as duas dependem do resultado uma da outra.

Saídas (antes e depois da correção)

```
IMPASSE: 2 threads presas.
A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

Pois ambas necessitam do resultado uma da outra. Não conseguindo finalizar sem que a outra finalize primeiro.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

impede que cada processo fique esperando por um recurso que está sendo segurado por outro processo que, por sua vez, espera pelo primeiro.
Sistema bancário com transferências entre contas.

Imagine duas transferências simultâneas:

Transferência 1: da conta A para a conta B.

Transferência 2: da conta B para a conta A.

Se a primeira operação bloquear A e depois tentar bloquear B, enquanto a segunda bloquear B e depois tentar bloquear A, poderá ocorrer um deadlock.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com 3 trabalhadores: cada um executa 2 tarefas. Como trabalham em paralelo, esperamos cerca de 200 ms.

Com 1 trabalhador: ele executa as 6 tarefas em sequência. Esperamos cerca de 600 ms.

Com 6 trabalhadores: cada trabalhador executa uma tarefa. Esperamos cerca de 100 ms.

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

```
const trabalhadores = 6                      // mude para 1 e depois para 6

func trabalhador(id int, tarefas ←chan int, wg *sync.WaitGroup) {
    defer wg.Done()
    for t := range tarefas {                 // recebe tarefas até o canal ser fechado
        time.Sleep(100 * time.Millisecond)   // simula um trabalho de 100 ms
        fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
    }
}
```

```
392 ms
trabalhador 5 fez a tarefa 6
trabalhador 2 fez a tarefa 3
trabalhador 6 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 3 fez a tarefa 4
trabalhador 4 fez a tarefa 5
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

O programa executa 6 tarefas de 100 ms cada. Com 1 trabalhador, leva cerca de 600 ms, pois ele faz todas as tarefas sozinho. Com 3 trabalhadores, leva cerca de 200 ms, pois cada um faz 2 tarefas. Com 6 trabalhadores, leva cerca de 100 ms, pois cada um faz uma tarefa ao mesmo tempo.
O programa não precisa de travas porque usa canais para distribuir as tarefas entre os trabalhadores, evitando que eles precisem modificar diretamente a mesma informação.

Saídas (antes e depois da troca)

```
}

Exit 1
fatal error: all goroutines are asleep - deadlock!
```

```
goroutine 1 [chan send]:
main.main()
        /box/main.go:9 +0x38
exit status 2
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque o canal make(chan int) não tem espaço para armazenar mensagens. Quando o programa executa ch <- 1, ele fica esperando que outra parte do programa receba o valor, mas isso não acontece, pois o recebimento só seria executado na linha seguinte. Assim, ocorre um bloqueio.
Ao usar make(chan int, 1), o canal passa a ter espaço para armazenar uma mensagem. Dessa forma, o valor 1 fica guardado no canal até ser recebido por <-ch, e o programa consegue continuar normalmente.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Acredito que o programa não vai compilar, porque o Rust não permite que uma thread altere uma variável de outra thread sem garantir que o acesso seja seguro.

Saídas (o erro e o resultado com move)

```
error[E0373]: closure may outlive the current function, but it borrows `contador`, which is
owned by the current function.

warning: value captured by `contador` is never read.
0
```

Saída do programa

```
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Com move, a thread recebe uma cópia do valor da variável contador, que começa em 0. A thread altera sua própria cópia para 1, mas a variável original da função main continua valendo 0. Por isso, ao executar println!("{}", contador), o programa imprime 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o erro de condição de corrida só é percebido quando o programa é executado: o resultado pode ser menor que o esperado porque as duas threads alteram o contador ao mesmo tempo. Em Rust, o compilador tenta identificar esse tipo de problema antes da execução, impedindo acessos simultâneos inseguros à variável.

A vantagem do Rust é oferecer mais segurança antes de o programa rodar. O custo é que o programador precisa escrever um código mais cuidadoso, usando recursos como Mutex e Arc para compartilhar dados entre threads. Já em Java, é mais simples começar, mas o programador precisa tomar cuidado para evitar erros que podem aparecer somente durante a execução.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

Em JavaScript, o código geralmente roda em uma única thread, usando um laço de eventos (event loop) para organizar as tarefas. Isso evita que duas partes do código alterem uma variável ao mesmo tempo, como acontece em uma condição de corrida entre threads. Porém, se uma tarefa demorar muito, ela bloqueia a execução das outras até terminar. Por exemplo, se um cálculo muito pesado estiver rodando, a página pode parar de responder aos cliques temporariamente.

Saída do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A ordem é A, D, C, B e E. A e D são executados imediatamente. C vem depois porque é uma promessa, que tem prioridade sobre o setTimeout. B, mesmo com tempo 0, precisa esperar sua vez no laço de eventos. E aparece por último porque a função ocupado(300) bloqueia a thread por cerca de 300 ms.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer atrasou porque o cálculo pesado bloqueou a única thread por cerca de 300 ms. Em um servidor Node.js, isso pode atrasar as respostas de todos os usuários. Para evitar isso, podemos usar Worker Threads para executar cálculos pesados sem bloquear a thread principal.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode ocorrer uma condição de corrida, fazendo com que duas pessoas comprem o mesmo ingresso. Para evitar isso, usamos uma transação no banco de dados ou um bloqueio, garantindo que apenas uma compra seja confirmada.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi perceber que duas tarefas executadas ao mesmo tempo podem causar erros, e que cada linguagem tem maneiras diferentes de evitar isso.